Практическая работа 2

Баев Ярослав

1. Ответы на вопросы

Вопрос 1

В функции send_reliable используется TCP. Это видно по типу sock_stream и вызову connect(), который устанавливает соединение с сервером

Вопрос 2

В функции send_fast используется UDP. Это видно по типу sock_dgram и отправке данных через sendto(). Для UDP отдельное соединение не устанавливается

Вопрос 3

Данные передаются в формате json. Это подтверждает строка json.dumps(message, ensure_ascii=False). Она преобразует данные в текст перед отправкой на сервер

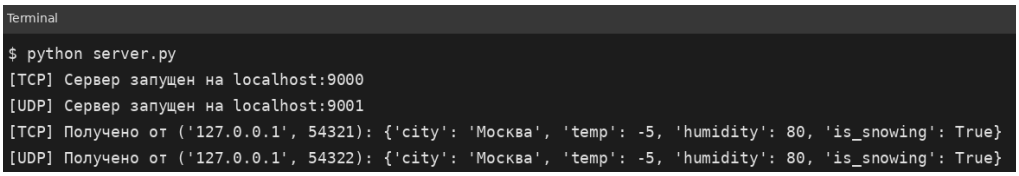
Вопрос 4

Сервер попробует разобрать полученные данные как json. Обычная строка hello не является правильным json, поэтому сервер выведет сообщение об ошибке и отправит ответ со статусом error и причиной invalid json

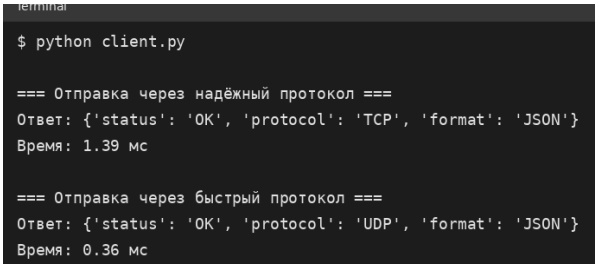
2. Запуск сервера и клиента

Сначала запускается server.py в одном окне терминала. Затем в другом окне запускается client.py. При успешной работе клиент получает ответы от сервера по TCP и UDP со статусом OK

Скриншот 1 - Окно сервера



Скриншот 2 - Окно клиента



По результатам запуска клиент получил ответ по обоим протоколам. Сервер принял данные о погоде: город Москва, температура −5, влажность 80 и наличие снега

3. Тестирование

Случай 1 Сервер выключен, клиент TCP

Если сервер выключен, клиент не может установить TCP-соединение. Обычно появляется ошибка подключения, например ConnectionRefusedError. Это происходит потому, что на указанном порту нет работающего сервера. В обычном запуске эта ошибка не обработана в коде клиента

Случай 2 Сервер выключен, клиент UDP

UDP-клиент отправляет сообщение, но сервер не отвечает. Клиент ждёт ответ до двух секунд, потому что в коде задано ограничение времени ожидания 2.0 секунды. После этого клиент возвращает статус TIMEOUT. UDP не проверяет заранее, работает ли получатель

Случай 3 Клиент отправляет не JSON

При отправке байтов hello сервер не может разобрать сообщение как JSON. В окне сервера появляется сообщение о том, что JSON не удалось разобрать. Клиент получает ответ со статусом ERROR и причиной invalid JSON. После проверки исходную строку отправки нужно вернуть

Случай 4 Сравнение форматов

При запуске format_check.py размер JSON составил 72 байта, а XML — 111 байт. В этом примере JSON компактнее, потому что в XML повторяются открывающие и закрывающие теги. При этом восстановленные данные из XML содержат значения в виде строк, а JSON сохраняет число и логическое значение как соответствующие типы

4. Вывод

В ходе работы разобрал код клиент-серверного приложения и проверил, как передаются данные. TCP подходит, когда важна надёжная передача и подтверждение соединения. UDP проще и может быть быстрее, но ответ может не прийти. Для передачи данных в этом приложении я бы выбрал JSON, потому что в данном примере он занимает меньше места и его удобно обрабатывать в Python